Chapter 2

Processes and Threads

2.1 Processes
2.2 Threads
2.3 Inter-process communication
2.4 Classical IPC problems
2.5 Scheduling

Inter-process Communication

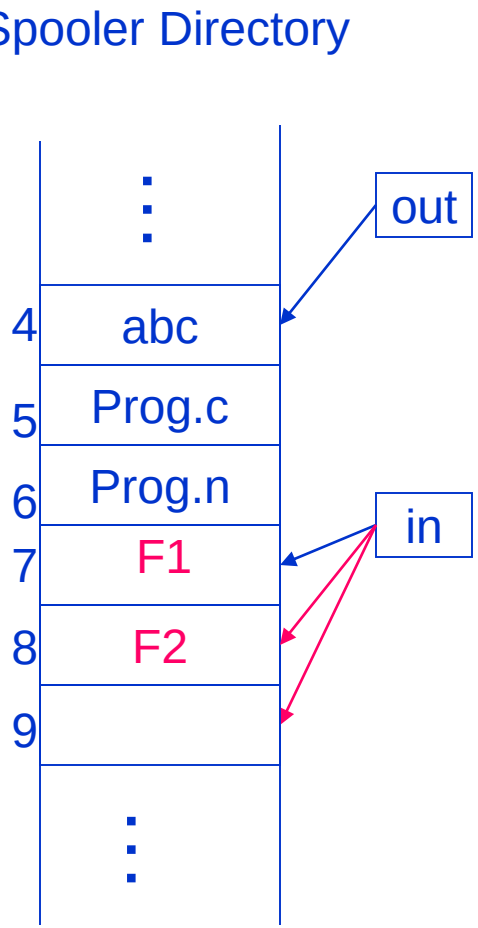
- Three issues are involved in inter-process communication (IPC)
  1. How one process can pass information to another.
  2. Resource shared (e.g. printer)
     - How to make sure two or more processes do not get into each other's way when engaging in critical activities
  3. Process cooperation
     - Proper sequencing when dependencies are present

Spooling Example: Correct

Process 1

int next_free;

1. next_free = in;
2. Stores F1 into next_free;
3. in=next_free+1



Process 2

int next_free;

4. next_free = in
5. Stores F2 into next_free;
6. in=next_free+1

Spooling Example: Races

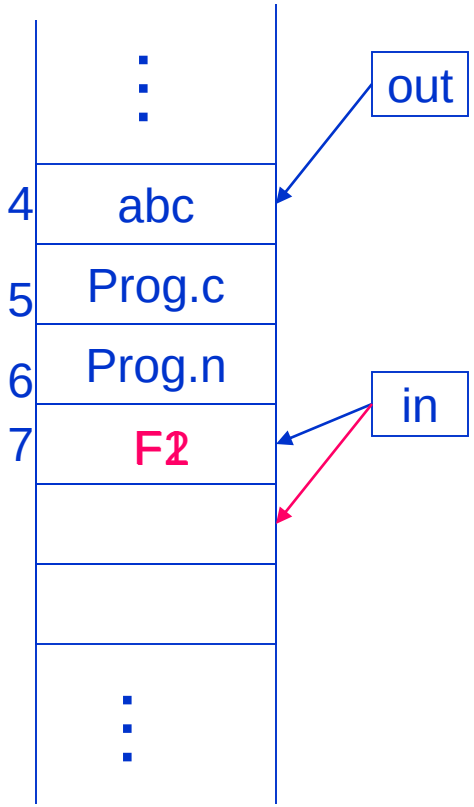
Process 1

int next_free;

1. next_free = in;
   /* value: 7 */

5. Stores F1 into next_free;
6. in=next_free+1

Shared memory



Process 2

int next_free;

2. next_free = in
   /* value: 7 */
3. Stores F2 into next_free;
4. in=next_free+1

Race conditions

Race conditions are situations in which several processes access shared data and the final result depends on the order of operations.

The key to avoid race conditions is to prohibit more than one process from reading and writing the shared data at the same time.

Mutual exclusion

Some way of making sure that if one process is using a shared variable or file, the other processes will be excluded from doing the same thing.

- Critical Resource(临界资源)
  - 一次仅允许一个进程访问的资源称为临界资源
  - 临界资源包括：
    - 硬件资源：输入机、打印机等
    - 软件资源：变量、表格、队列、文件等

- Critical Region (Critical Section)
  - The part of the program where the critical resource is accessed is called critical region or critical section.

If we could arrange matters such that no two processes were ever in their critical regions at the same time, we could avoid races.

P_1

```
a := a +1
print (a)
```

P_2

```
a := a -1
print (a)
```

P_3

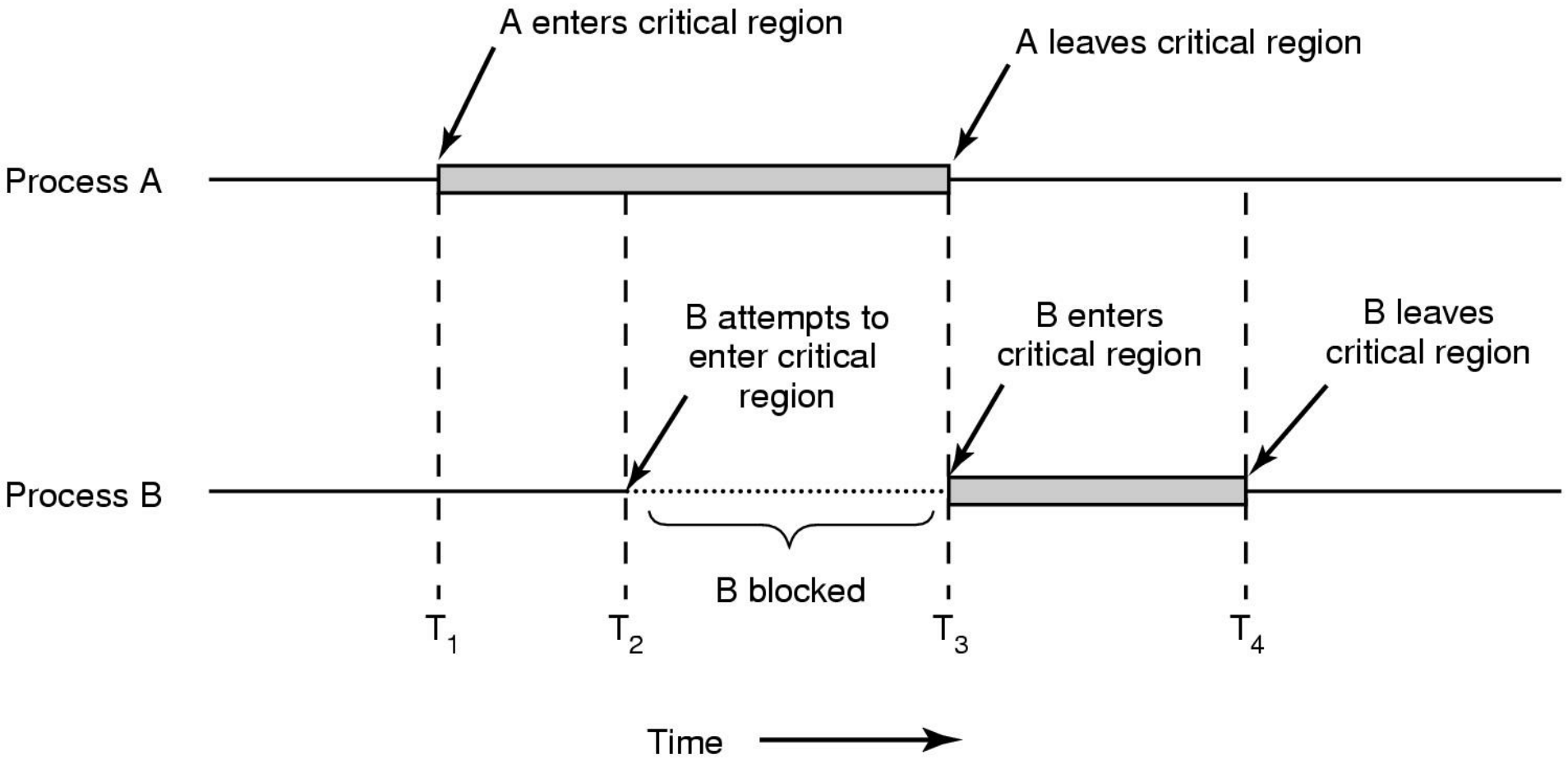
```
If a < 0
then
a := a +1
else
a:= a-1
```

Mutual exclusion

```
entry section
  critical section
exit section
  remainder section
```

Critical Region Requirement

- Four conditions to provide mutual exclusion
  1. No two processes simultaneously in critical region
  2. No assumptions made about speeds or numbers of CPUs
  3. No process running outside its critical region may block another process
  4. No process must wait forever to enter its critical region



Mutual exclusion using critical regions

Mutual Exclusion

- Possible Solutions
  - Disabling Interrupts
  - Lock Variables
  - Strict Alternation
  - Peterson’s solution
  - TSL
  - Sleep and Wakeup

Solution 1 - Disabling Interrupts

- How does it work?
  - Disable all interrupts just after entering a critical section and re-enable them just before leaving it.
- Why does it work?
  - With interrupts disabled, no clock interrupts can occur. (The CPU is only switched from one process to another as a result of clock or other interrupts, and with interrupts disabled, no switching can occur.)
- Problems:
  - What if the process forgets to enable the interrupts?
  - Multiprocessor? (disabling interrupts only affects one CPU)
- Only used inside OS

Solution 2 - Lock Variable

```
shared int lock = 0;
/* entry_code: execute before entering critical section */
while (lock != 0) /* do nothing */ ;
lock = 1;

    - critical section -

/* exit_code: execute after leaving critical section */
lock = 0;
```

This solution may violate property 1. If a context switch occurs after one process executes the while statement, but before setting lock = 1, then two (or more) processes may be able to enter their critical sections at the same time.

Solution 3 - Strict Alternation

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

(a)

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

(b)

This solution may violate property 3. Since the processes must strictly alternate entering their critical sections, a process wanting to enter its critical section twice in a row will be blocked until the other process decides to enter (and leave) its critical section.

Solution 4 - Peterson's

process i

```
enter_region ( i );
Critical region
leave_region ( i );
Noncritical region
```

Solution 4 - Peterson's

```
#define FALSE  0
#define TRUE   1
#define N      2                        /* number of processes */

int turn;                               /* whose turn is it? */
int interested[N];                      /* all values initially 0 (FALSE) */

void enter_region(int process);         /* process is 0 or 1 */
{
     int other;                         /* number of the other process */

     other = 1 – process;               /* the opposite of process */
     interested[process] = TRUE;        /* show that you are interested */
     turn = process;                    /* set flag */
     while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process)          /* process: who is leaving */
{
     interested[process] = FALSE;       /* indicate departure from critical region */
}
```

Peterson's solution for achieving mutual exclusion

Hardware solution 5: Test-and-Set Locks (TSL)

The hardware must support a special instruction, tsl, which does 2 things in a single atomic action:

tsl register, flag

(a) copy a value in memory (flag) to a CPU register

(b) set flag to 1.

Test-and-Set Locks (TSL)

```
enter_region:
        TSL REGISTER,LOCK                | copy lock to register and set lock to 1
        CMP REGISTER,#0                  | was lock zero?
        JNE enter_region                 | if it was non zero, lock was set, so loop
        RET | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0                     | store a 0 in lock
        RET | return to caller
```

Entering and leaving a critical region using the TSL instruction

返回

Test-and-Set Locks (TSL)

```
enter_region:
        MOVE REGISTER,#1          | put a 1 in the register
        XCHG REGISTER,LOCK        | swap the contents of the register and lock variable
        CMP REGISTER,#0           | was lock zero?
        JNE enter_region          | if it was non zero, lock was set, so loop
        RET                       | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0              | store a 0 in lock
        RET                       | return to caller
```

Entering and leaving a critical region using the XCHG instruction.

Mutual Exclusion with Busy Waiting

- The last two solutions, 4 and 5, require BUSY-WAITING; that is, a process executing the entry code will sit in a tight loop using up CPU cycles, testing some condition over and over, until it becomes true.

- Busy-waiting may lead to the PRIORITY-INVERSION PROBLEM if simple priority scheduling is used to schedule the processes.

Mutual Exclusion with Busy Waiting

- Example: Test-and-set Locks:

P0 (low) - in cs -x
|
context
switch
|
P1 (high) -----tsl-cmp-jnz-tsl... x-tsl-cmp... x-... forever.

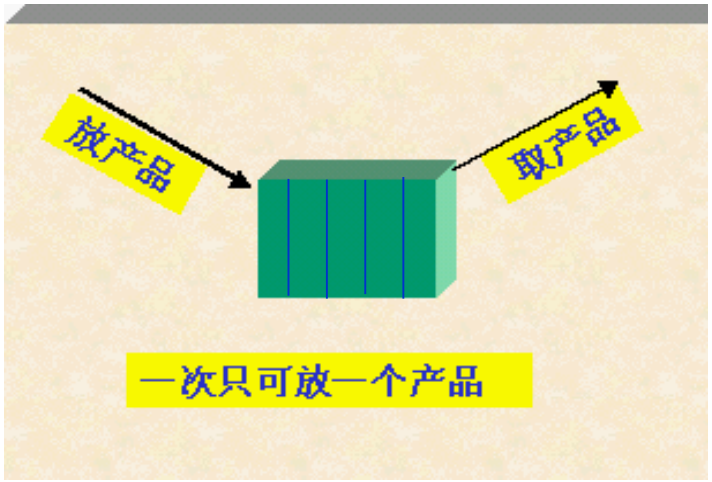
- Note, since priority scheduling is used, P1 will keep getting scheduled and waste time doing busy-waiting. :-(
- Thus, we have a situation in which a low-priority process is blocking a high-priority process, and this is called PRIORITY-INVERSION.

Sleep and Wakeup

- Solution of previous problems: sleep and wakeup (busy waiting)
  - Block instead of busy waiting when it is not allowed to enter the Critical Region (sleep)
  - Wakeup when it is OK to retry entering the critical region

Producer-Consumer Problem (Bounded-buffer problem)

- Consider two processes share a circular buffer that can hold N items.
- Producers add items to the buffer and Consumers remove items from the buffer.
- The Producer-Consumer Problem is to restrict access to the buffer so correct executions result.



Sleep and Wakeup

```
#define N 100                                   /* number of slots in the buffer */
int count = 0;                                  /* number of items in the buffer */

void producer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          item = produce_item( );               /* generate next item */
          if (count == N) sleep( );             /* if buffer is full, go to sleep */
          insert_item(item);                    /* put item in buffer */
          count = count + 1;                    /* increment count of items in buffer */
          if (count == 1) wakeup(consumer);     /* was buffer empty? */
     }
}


void consumer(void)
{
     int item;

     while (TRUE) {                             /* repeat forever */
          if (count == 0) sleep( );             /* if buffer is empty, got to sleep */
          item = remove_item( );                /* take item out of buffer */
          count = count – 1;                    /* decrement count of items in buffer */
          if (count == N – 1) wakeup(producer); /* was buffer full? */
          consume_item(item);                   /* print item */
     }
}
```

Producer-consumer problem with fatal race condition

Semaphores (E.W. Dijkstra,1965)

- A SEMAPHORE, S, is a structure consisting of two parts:

  (a) an integer counter, COUNT

  (b) a queue of pids of blocked processes, Q

- That is,

```
struct sem_struct {
        int count;
        queue Q;
} semaphore;

semaphore S;
```

Semaphores (信号量)

- A semaphore count represents count number of abstract resources
  - Counting semaphores: 0..N
  - Binary semaphores: 0,1
- There are 2 operations on semaphores
  - The Down (P) operation is used to acquire a resource and decrements count.
  - The Up (V) operation is used to release a resource and increments count.
- Any semaphore operation is indivisible, must be executed atomically. (what is primitive?)

Semaphores

- Suppose that P is the process making the down and up system calls. The operations are defined as follows:

P操作 DOWN(S):

```
S.count = S.count - 1;   //apply for a resource
if (S.count < 0)         //if  no resource
      block(P);
```

(a) enqueue the pid of P in S.Q,
(b) block process P (remove the pid from the ready queue), and
(c) pass control to the scheduler.

执行一次P操作后，若S.count<0，则|S.count|等于Q队列中等待S资源的进程数。

Semaphores

V操作 UP(S):

```
S.count = S.count + 1;      //release a resource
if ( S.count<= 0 )          // have process blocked
    wakeup(P) for some process P in S.Q;
```

(a) remove a pid from S.Q (the pid of P),
(b) put the pid in the ready queue, and
(c) pass control to the scheduler.

执行一次V操作后，若S.count≤0，则表明Q队列中仍有因等待S资源而被阻塞的进程，所以需要唤醒(wakeup)Q队列中的进程。

The producer-consumer problem using semaphores

```
#define N 100                                   /* number of slots in the buffer */
typedef int semaphore;                          /* semaphores are a special kind of int */
semaphore mutex = 1;                            /* controls access to critical region */
semaphore empty = N;                            /* counts empty buffer slots */
semaphore full = 0;                             /* counts full buffer slots */

void producer(void)
{
     int item;

     while (TRUE) {                             /* TRUE is the constant 1 */
          item = produce_item( );                /* generate something to put in buffer */
          down(&empty);                          /* decrement empty count */
          down(&mutex);                          /* enter critical region */
          insert_item(item);                     /* put new item in buffer */
          up(&mutex);                            /* leave critical region */
          up(&full);                             /* increment count of full slots */
     }
}


void consumer(void)
{
     int item;

     while (TRUE) {                             /* infinite loop */
          down(&full);                           /* decrement full count */
          down(&mutex);                          /* enter critical region */
          item = remove_item( );                 /* take item from buffer */
          up(&mutex);                            /* leave critical region */
          up(&empty);                            /* increment count of empty slots */
          consume_item(item);                    /* do something with the item */
     }
}
```

- In conclusion, we use semaphores in two different ways:
  - mutual exclusion (mutex);
  - process synchronization (full, empty).

Mutex（互斥量): Binary Semaphore

- Simplified version of semaphore
- Mutex is used for mutual exclusion problem
- A variable with only 2 states
  - Lock
  - Unlock

Mutual Exclusion Problem using Mutex

```
semaphore mutex = 1;   /* set mutex.count = 1 */

DOWN(mutex);
        - critical section -
UP(mutex);
```

- For two parallel processes, the value of mutex is 1, 0, -1
  For N parallel processes, the value range of mutex is 1~ -(N-1)

Mutex Implementation using TSL

TSL的enter_region

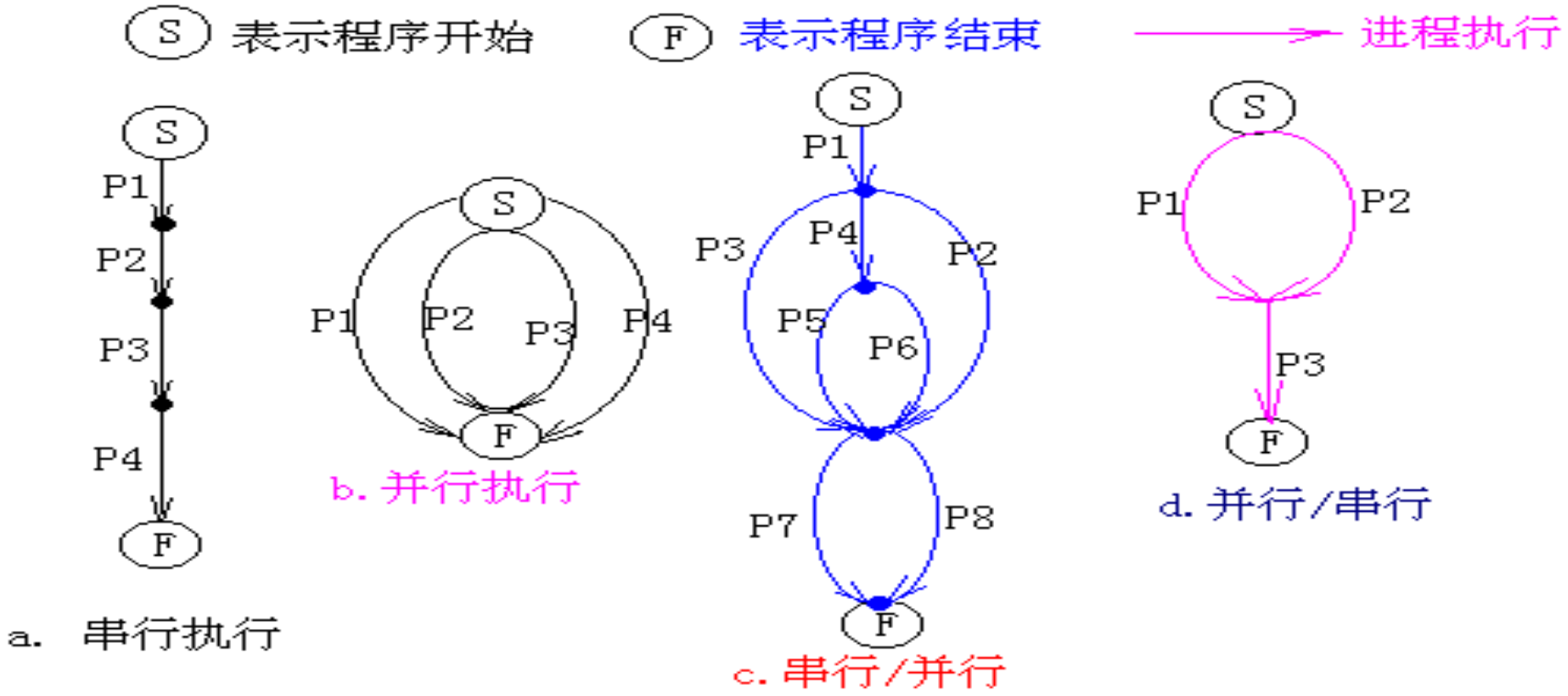
```
mutex_lock:
        TSL REGISTER,MUTEX          | copy mutex to register and set mutex to 1
        CMP REGISTER,#0             | was mutex zero?
        JZE ok                      | if it was zero, mutex was unlocked, so return
        CALL thread_yield           | mutex is busy; schedule another thread
        JMP mutex_lock              | try again later
ok:     RET | return to caller; critical region entered

mutex_unlock:
        MOVE MUTEX,#0               | store a 0 in mutex
        RET | return to caller
```

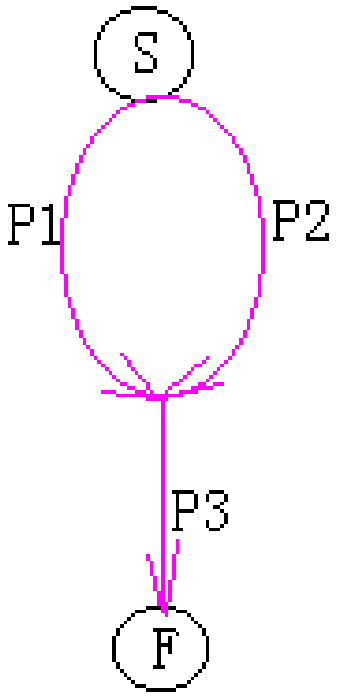
Implementation of *mutex_lock* and *mutex_unlock*

Using Semaphores

- Process Synchronization: Order process execution

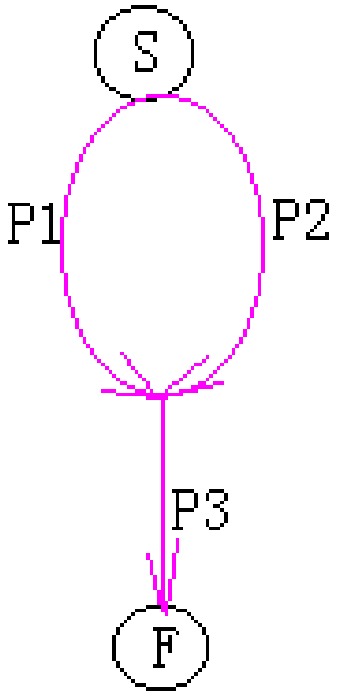


Using Semaphores



- Suppose we have 3 processes: P1, P2, and P3. P1 and P2 must finish executing before P3.
- The processes may be synchronized using semaphores:
  - S1=0: P1 still unfinished;
  - S2=0: P2 still unfinished;

Using Semaphores



P1()	P2()	P3()
{	{	{
……	……	DOWN(S1);
UP(S1);	UP(S2);	DOWN(S2);
}	}	……
		}

讨论：

1) 信号量的物理含义

S.count>0表示有S.count个资源可用
S.count=0表示无资源可用
S.count<0则|S.count|表示S等待队列中的进程个数

P(S)：表示申请一个资源
V(S)：表示释放一个资源。信号量的初值应该大于等于0

2) P,V操作必须成对出现，有一个P操作就一定有一个V操作

- 当为互斥操作时，它们同处于同一进程；
  当为同步操作时，则不在同一进程中出现。
- 如果P(S1)和P(S2)两个操作在一起，那么P操作的顺序至关重要，一个同步P操作与一个互斥P操作在一起时同步P操作在互斥P操作前；
- 而两个V操作的顺序无关紧要。

3) 信号量同步的缺点

用信号量可实现进程间的同步，但由于信号量的控制分布在整个程序中，其正确性分析很困难。

4) 引入管程

- 1973年，Hoare和Hanson提出一种高级同步原语——管程；其基本思想是把信号量及其操作原语封装在一个对象内部。
- 管程是管理进程间同步的机制，它保证进程互斥地访问共享变量，并方便地阻塞和唤醒进程。
- 管程可以函数库的形式实现。相比之下，管程比信号量好控制。

Monitors

- A **monitor** is a collection of procedures, variables, and data structures that can only be accessed by one process at a time (for the purpose of mutual exclusion).
- To allow a process to wait within the monitor, a **condition variable** must be declared, as **condition x, y;**
- Condition variable can only be used with the operations **wait** and **signal** (for the purpose of synchronization).
  - The operation

    wait(x);

    means that the process invoking this operation is suspended until another process invokes

    signal(x);
  - The **signal(x)** operation resumes exactly one suspended process. If no process is suspended, then the **signal** operation has no effect.

Monitors

```
monitor example
        integer i;
        condition c;

        procedure producer( );

        .
        .
        .
        end;

        procedure consumer( );

        .
        .
        .
        end;
end monitor;
```

Example of a monitor

Monitors

```
monitor ProducerConsumer
        condition full, empty;
        integer count;
        procedure insert(item: integer);
        begin
                if count = N then wait(full);
                insert_item(item);
                count := count + 1;
                if count = 1 then signal(empty)
        end;
        function remove: integer;
        begin
                if count = 0 then wait(empty);
                remove = remove_item;
                count := count - 1;
                if count = N - 1 then signal(full)
        end;
        count := 0;
end monitor;
```

```
procedure producer;
begin
        while true do
        begin
                item = produce_item;
                ProducerConsumer.insert(item)
        end
end;
procedure consumer;
begin
        while true do
        begin
                item = ProducerConsumer.remove;
                consume_item(item)
        end
end;
```

Outline of producer-consumer problem with monitors

- only one monitor procedure active at one time
- buffer has *N* slots

Message passing

Possible Approaches of IPC:

1) Shared memory
2) Shared file mode;

pipe: is a shared file

- One end is for reading and one end is for writing.

3) Message passing:

primitive: send and receive

- Assign each process a unique address such as addr. Then, send messages directly to the process:

send(addr, msg);
recv(addr, msg);

- Use mailboxes:

send(mailbox, msg);
recv(mailbox, msg);

Message Passing

```
#define N 100                                   /* number of slots in the buffer */

void producer(void)
{
      int item;
      message m;                                /* message buffer */

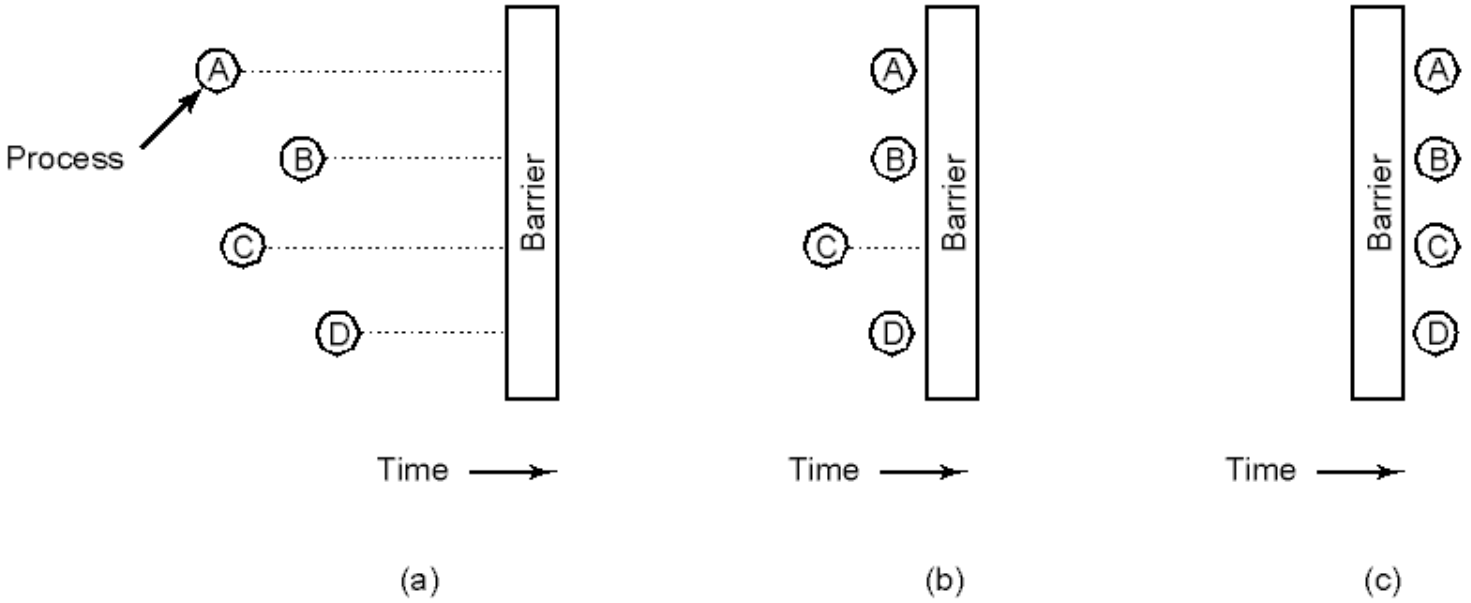
      while (TRUE) {
            item = produce_item( );             /* generate something to put in buffer */
            receive(consumer, &m);              /* wait for an empty to arrive */
            build_message(&m, item);            /* construct a message to send */
            send(consumer, &m);                 /* send item to consumer */
      }
}

void consumer(void)
{
      int item, i;
      message m;

      for (i = 0; i < N; i++) send(producer, &m);  /* send N empties */
      while (TRUE) {
            receive(producer, &m);              /* get message containing item */
            item = extract_item(&m);            /* extract item from message */
            send(producer, &m);                 /* send back empty reply */
            consume_item(item);                 /* do something with the item */
      }
}
```

The producer-consumer problem with N messages

Barriers



- Use of a barrier
  a) processes approaching a barrier
  b) all processes but one blocked at barrier
  c) last process arrives, all are let through
- Example: Parallel matrix multiplication

Classical IPC Problems

- These problems are used for testing every newly proposed synchronization scheme:
  - Bounded-Buffer (Producer-Consumer) Problem
  - Dining-Philosophers Problem
  - Readers and Writers Problem

Dining Philosophers

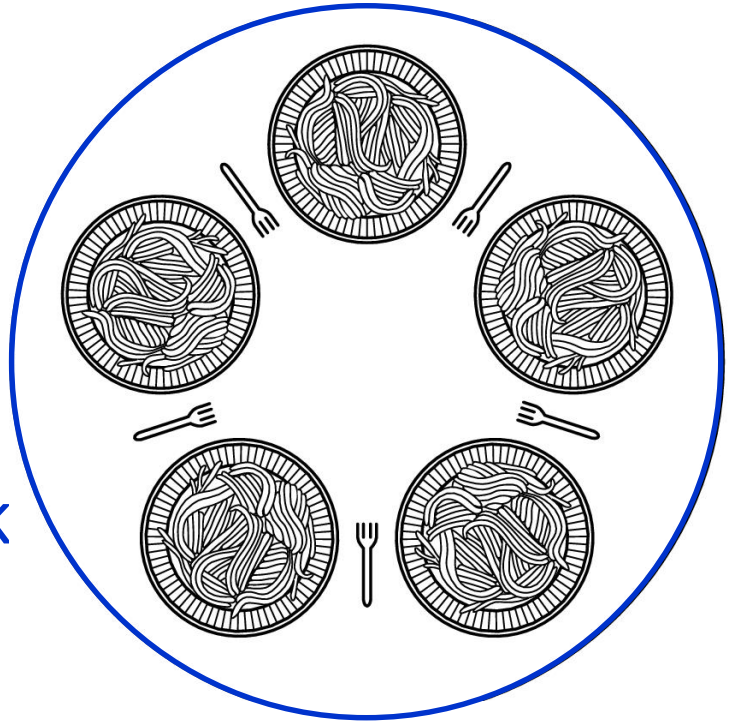
- Dining Philosophers Problem [Dijkstra, 1965]:

  Problem:

  - Five philosophers are seated around a table. There is one fork between each pair of philosophers.
  - Each philosopher needs to grab the two adjacent forks in order to eat.
  - Philosophers alternate between eating and thinking.
  - They only eat for finite periods of time.

Dining Philosophers

- Philosophers eat/think
- Eating needs 2 forks
- Pick one fork at a time
- How to prevent deadlock



Dining Philosophers

```
#define N 5                                     /* number of philosophers */

void philosopher(int i)                         /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                               /* philosopher is thinking */
        take_fork(i);                           /* take left fork */
        take_fork((i+1) % N);                   /* take right fork; % is modulo operator */
        eat( );                                 /* yum-yum, spaghetti */
        put_fork(i);                            /* put left fork back on the table */
        put_fork((i+1) % N);                    /* put right fork back on the table */
    }
}
```

A nonsolution to the dining philosophers problem

Dining Philosophers

- Problem:
  - Suppose all philosophers execute the first DOWN operation, before any have a chance to execute the second DOWN operation; that is, they all grab one fork. Then, deadlock will occur and no philosophers will be able to proceed. This is called a CIRCULAR WAIT.
- Other Solutions:
  - Only allow up to four philosophers to try grabbing their forks.
  - Asymmetric solution: Odd numbered philosophers grab their left fork first, whereas even numbered philosophers grab their right fork first.
  - Protect the five statements following *think()* by mutex
  - Pick-up the forks only if both are available. See Fig. 2-46.

Dining Philosophers

```
#define N                5          /* number of philosophers */
#define LEFT             (i+N−1)%N  /* number of i's left neighbor */
#define RIGHT            (i+1)%N    /* number of i's right neighbor */
#define THINKING         0          /* philosopher is thinking */
#define HUNGRY           1          /* philosopher is trying to get forks */
#define EATING           2          /* philosopher is eating */
typedef int semaphore;              /* semaphores are a special kind of int */
int state[N];                       /* array to keep track of everyone's state */
semaphore mutex = 1;                /* mutual exclusion for critical regions */
semaphore s[N];                     /* one semaphore per philosopher */

void philosopher(int i)             /* i: philosopher number, from 0 to N−1 */
{
    while (TRUE) {                  /* repeat forever */
        think( );                   /* philosopher is thinking */
        take_forks(i);              /* acquire two forks or block */
        eat( );                     /* yum-yum, spaghetti */
        put_forks(i);               /* put both forks back on table */
    }
}
```

Solution to dining philosophers problem (part 1)

Dining Philosophers

```
void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
      down(&mutex);                     /* enter critical region */
      state[i] = HUNGRY;                /* record fact that philosopher i is hungry */
      test(i);                          /* try to acquire 2 forks */
      up(&mutex);                       /* exit critical region */
      down(&s[i]);                      /* block if forks were not acquired */
}

void put_forks(i)                       /* i: philosopher number, from 0 to N–1 */
{
      down(&mutex);                     /* enter critical region */
      state[i] = THINKING;              /* philosopher has finished eating */
      test(LEFT);                       /* see if left neighbor can now eat */
      test(RIGHT);                      /* see if right neighbor can now eat */
      up(&mutex);                       /* exit critical region */
}

void test(i)                            /* i: philosopher number, from 0 to N–1 */
{
      if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
            state[i] = EATING;
            up(&s[i]);
      }
}
```

Solution to dining philosophers problem (part 2)

Readers and Writers Problem

- The readers and writers problem models access to a shared database.
- Rules:
  - Multiple readers can read the data simultaneously
  - Only one writer can write the data at any time
  - A reader and a writer cannot in critical section together.

	Reader	Writer
Reader	OK	No
Writer	No	No

Readers and Writers Problem

- One solution (reader preference):
  - Suppose new reader come: if writer is waiting and some reader is reading, then read ok.
  - Suppose new writer come: if no reader or writer, it can write, else wait.
- One variation of the problem, called weak readers preference, is to suspend the incoming readers as long as a writer is waiting.

Readers and Writers Problem

```
typedef int semaphore;                  /* use your imagination */
semaphore mutex = 1;                    /* controls access to 'rc' */
semaphore db = 1;                       /* controls access to the database */
int rc = 0;                             /* # of processes reading or wanting to */

void reader(void)
{
      while (TRUE) {                    /* repeat forever */
            down(&mutex);               /* get exclusive access to 'rc' */
            rc = rc + 1;                /* one reader more now */
            if (rc == 1) down(&db);     /* if this is the first reader ... */
            up(&mutex);                 /* release exclusive access to 'rc' */
            read_data_base( );          /* access the data */
            down(&mutex);               /* get exclusive access to 'rc' */
            rc = rc − 1;                /* one reader fewer now */
            if (rc == 0) up(&db);       /* if this is the last reader ... */
            up(&mutex);                 /* release exclusive access to 'rc' */
            use_data_read( );           /* noncritical region */
      }
}


void writer(void)
{
      while (TRUE) {                    /* repeat forever */
            think_up_data( );           /* noncritical region */
            down(&db);                  /* get exclusive access */
            write_data_base( );         /* update the data */
            up(&db);                    /* release exclusive access */
      }
}
```

A solution to the readers and writers problem

Summary

- Race condition, Critical region and mutual exclusion
- Mutual exclusion using busy waiting
  - Peterson's Solution
  - TSL
- Sleep and Wakeup
- Semaphores
- Monitor
- Barrier
- Passing Message
- Classic synchronization problems
  - Dining-Philosophers Problem
  - Readers and Writers Problem

Thank you!

